

Projet Creative-Yumland

Arjuna Jeevananthan et Parrukan Kalaichelvam

Phase 1 – HTML + CSS

Nous avons défini notre charte graphique en choisissant un ensemble de couleurs cohérent. Nous avons opté pour une palette dorée car nous estimions qu'elle correspondait bien au thème du restaurant, à savoir un restaurant de luxe à l'italienne. Le fond principal des pages est en blanc crème (rgb(250, 245, 235)). Les éléments d'en-tête et de navigation utilisent un dégradé doré allant du rgb(184, 148, 58) au rgb(192, 173, 125), avec du texte blanc pour assurer la lisibilité. Les accents et liens sont dans un or plus soutenu (rgb(191, 167, 106) et rgb(212, 175, 55)), tandis que le texte courant est en brun foncé (rgb(75, 59, 47) ou rgb(127, 103, 6)) pour rester dans la tonalité chaude de l'ensemble. Les fonds de certains blocs secondaires utilisent des variantes crème et sable (rgb(248, 245, 235), rgb(236, 223, 187), rgb(194, 177, 134)). L'ensemble de ces couleurs a été intégré dans un fichier CSS unique partagé par toutes les pages du site.

Nous avons réalisé les neuf pages demandées dans le cahier des charges, chacune dans un fichier HTML séparé : la page d'accueil avec le nom du restaurant, une zone de recherche et une liste de plats, la page de présentation des produits avec barre de recherche et filtres par catégorie, la page d'inscription, la page de connexion, la page de profil, la page administrateur affichant la liste des utilisateurs, la page restaurateur pour visualiser les commandes en attente et en cours, la page livreur, et enfin la page de notation.

Arjuna Jeevananthan

Durant la phase 1, j'ai eu la charge de la réalisation de plusieurs pages principales : la page d'accueil, la page de présentation des produits, la page administration et la page profil utilisateur. Le travail s'est déroulé dans de bonnes conditions grâce à une répartition claire des tâches au sein du groupe dès le début du projet. Ces pages ont été développées en HTML en utilisant le fichier CSS commun afin d'assurer une cohérence graphique sur l'ensemble du site. La principale difficulté rencontrée concernait l'alignement des textes dans le header en CSS. J'ai utilisé l'IA pour trouver une solution et me suis également documenté sur les différentes propriétés CSS pour mieux maîtriser ce langage. En dehors de ce point, aucune difficulté technique majeure n'a été rencontrée.

Parrukan Kalaichelvam

Lors de la première phase, j'ai conçu cinq pages essentielles au fonctionnement de l'application : la page d'inscription, la page de connexion, le système d'avis, ainsi que les pages de gestion des commandes et des livraisons. J'ai pris en charge l'écriture complète du code HTML de manière autonome. Pour la partie CSS, je me suis appuyé sur des ressources en ligne afin de parfaire l'identité visuelle du site, ce qui m'a permis de sélectionner des couleurs pertinentes et de structurer l'agencement des titres et des blocs pour offrir une expérience utilisateur fluide. Durant la première semaine, je me suis concentré sur les pages de formulaires : l'inscription, la connexion et le système d'avis. Durant la deuxième semaine, j'ai finalisé le développement avec les pages de gestion des commandes et des livraisons.

Phase 2 – PHP

La phase 2 nous a amenés à rendre le site fonctionnel côté serveur. Nous avons commencé par définir précisément notre modèle de données. Nous avons choisi de stocker toutes les données au format JSON dans un dossier /data/ séparé des scripts PHP, ce qui nous a paru plus simple à mettre en œuvre.

Nous avons créé quatre fichiers de données. Le fichier utilisateurs.json contient les informations de chaque utilisateur : identifiant, login, mot de passe hashé, rôle (client, restaurateur, admin ou livreur), nom, prénom, date de naissance, adresse, numéro de téléphone, date d'inscription, date de dernière connexion, statut (actif ou bloqué), niveau de remise et points de fidélité. Le fichier plats.json regroupe le catalogue des plats avec leur nom, description, prix, catégorie, allergènes, saveur, disponibilité et nombre de commandes. Le fichier menu.json décrit les menus préconfigurés avec leur liste de plats, leur prix total, le nombre de personnes minimum et leurs créneaux de disponibilité. Le fichier commande.json stocke chaque commande avec la liste des articles commandés, l'adresse de livraison, le type (sur place, à emporter ou en livraison), le statut, la date souhaitée et l'identifiant du livreur assigné.

Arjuna Jeevananthan

Durant la phase 2, j'ai développé les pages suivantes : la page d'inscription, la page de connexion, la page administrateur avec accès aux profils utilisateurs, la page panier, la page paiement avec intégration de l'API CYBank, et la page de retour paiement. Le principal problème technique rencontré en séance était lié à la présence en double du fichier fonctions.inc.php, ce qui empêchait le lancement du serveur. Une fois le doublon supprimé, le problème a été résolu. Concernant l'intégration de CYBank, la compréhension du mécanisme d'envoi et de retour des paramètres ainsi que le calcul de la valeur de contrôle md5 était complexe. J'ai utilisé l'IA pour m'aider à comprendre le fonctionnement, ce qui m'a permis de l'implémenter correctement. Du côté CSS, j'ai identifié un problème d'optimisation : pour chaque bouton de formulaire PHP nécessitant un alignement à droite, je créais une nouvelle classe CSS au lieu de réutiliser les classes existantes. Ce point a été corrigé. En dehors de ces points, le développement des autres pages s'est déroulé sans difficulté majeure.

Parrukan Kalaichelvam

Durant la phase 2, j'ai développé le côté restaurateur avec la liste détaillée des commandes et le détail d'une commande. J'ai également réalisé la partie livreur et la partie client, qui permet d'accéder à la page de profil, de consulter l'historique des commandes, et pour une commande récemment livrée, de laisser une note. J'ai rencontré des difficultés avec PHP, que j'ai surmontées en relisant le cours et en refaisant les TD. Pour la page livreur, j'avais les fonctionnalités de base mais il me manquait le CSS ; j'ai donc réutilisé ce qui avait déjà été fait, comme pour d'autres pages. J'ai également ajouté un système d'étoiles pour les avis : j'ai demandé à une IA comment procéder puis je l'ai implémenté moi-même. Je n'ai pas rencontré d'autres difficultés particulières.

Phase 3 – JavaScript

La première fonctionnalité mise en place est le changement de charte graphique avec un mode clair et un mode sombre. Un bouton disponible sur toutes les pages permet de basculer entre les deux thèmes sans recharger la page, en remplaçant dynamiquement l'attribut href de la balise <link> CSS via JavaScript. Le choix de l'utilisateur est sauvegardé dans un cookie. À chaque chargement de page, la valeur de ce cookie est vérifiée : si elle est absente ou incohérente, le mode clair par défaut est appliqué. La charte du mode sombre, définie dans le fichier darkstyle.css, adopte une palette violacée et profonde : le fond général passe en bleu nuit très foncé (rgb(26, 26, 46)), les blocs et cartes en violet sombre (rgb(55, 52, 90) et rgb(45, 43, 85)), les dégradés de navigation en violet moyen (rgb(45, 43, 85) à rgb(58, 58, 106)), le texte principal en gris clair (rgb(224, 224, 224)) et les accents en mauve clair (rgb(160, 143, 192) et rgb(201, 184, 232)).

Pour chaque formulaire existant, une validation JavaScript côté client a été mise en place. Sur la page de présentation des produits, des filtres et des tris ont été implémentés. Sur la page de profil, un utilisateur peut cliquer sur un bouton pour modifier ses informations personnelles (nom et prénom). Sur la page des commandes du restaurateur, ce dernier peut désormais changer le statut d'une commande. Un client peut également ajouter ou retirer des articles sur une commande déjà payée mais pas encore en préparation.

Arjuna Jeevananthan

Durant la phase 3, j'ai pris en charge le changement de charte graphique avec l'implémentation du mode clair et du mode sombre, en utilisant un cookie pour sauvegarder le choix de l'utilisateur et en permutant dynamiquement le fichier CSS sans rechargement de page. J'ai également mis en place les filtres asynchrones sur la page presentation.php ainsi que la modification asynchrone du profil client. Sur le page restaurateur, j'ai développé la gestion des changements de statut des commandes via des requêtes asynchrones. J'ai enfin implémenté la fonctionnalité permettant à un client de modifier une commande déjà payée avant qu'elle ne passe en préparation, avec recalcul automatique du montant et déclenchement d'un paiement complémentaire si nécessaire. Les difficultés rencontrées ont principalement porté sur les requêtes asynchrones. J'ai utilisé l'IA pour m'aider à comprendre le cours et avoir une idée de comment rendre fonctionnelles mes implémentations. Cela m'a permis d'avoir un fil conducteur à suivre : une fois le plan établi, je n'avais plus qu'à coder ce qui était demandé. Pour le mode clair/sombre, j'avais l'exemple fait en classe qui m'a grandement aidé.

Parrukan Kalaichelvam

Durant la phase 3, j'ai développé plusieurs fonctionnalités asynchrones ainsi que la validation des formulaires côté client.

J'ai réalisé la page avis.php, qui permet à un client de noter une commande récemment livrée en lui attribuant une note de livraison, une qualité de produit et un commentaire libre. Cette notation n'est possible qu'une seule fois par commande. J'ai également complété la page Livreur.php, qui permet au livreur de marquer une livraison comme effectuée ou abandonnée, avec une mise à jour immédiate de l'interface sans rechargement de page. Pour la partie administration, j'ai ajouté sur profil_version_admin.php la possibilité de bloquer ou débloquer un compte utilisateur.

Pour la validation des formulaires, j'ai modifié les pages Connexion.php et Inscription.php afin de vérifier les champs côté client avant l'envoi : format de l'email, longueur du mot de passe, format du numéro de téléphone. Si un champ est invalide, un message d'erreur s'affiche dans la page sans que la requête HTTP parte. J'ai également ajouté un compteur de caractères en temps réel pour le champ mot de passe, ainsi qu'un bouton permettant d'afficher ou de cacher le mot de passe.

Au cours de ce projet, j'ai rencontré des difficultés avec les requêtes asynchrones, notamment pour comprendre le fonctionnement de `onreadystatechange` et la différence entre `readyState` et `status`. En relisant le cours AJAX et en analysant les exemples fournis, j'ai réussi à les surmonter. Je n'ai pas rencontré d'autres difficultés particulières.